# Assignment 2

Tutors: Jun Wang, Yuhao Wu, Anjin Liu
Group members: (Ethan Yin 490427723, Nathan Truong 490473443, Numan Rsheidat 510403029 nrsh9805)

## Abstract

With the rise in deep learning and accessibility of GPUs training neural networks have become common place, however, performance still hinges on large, correctly labelled datasets. In the real world most human labelled datasets are riddled with incorrect labels. These noisy labels can lead to bias models unless explicitly addressed. In this report we evaluate three noise robust classifiers; forward learning, bootstrapping and co-teaching on three image datasets corrupted with class-conditional random label noise. We are given transition matrices for two of the datasets but for the third we developed a transition matrix estimator using a bootstrapping strategy. Each model was trained independently 10 times on a random 80/20 split of the training data and was evaluated on the clean test set using the top-1 accuracy score. Our study showed co-teaching performing the best on the two simpler gray scale datasets while bootstrapping performed the best on the complex coloured dataset.

## 1 Introduction

With the rise of deep learning techniques and GPUs to accelerate the learning of these complex models it has become much easier to train and build your own neural network. These models require large and accurately labelled datasets in order to perform well. However, in the real world from non-expert sources such as those from Amazon's Mechanical Turk crowd sourcing it can be hard to find cleanly labelled datasets. The errors and mismatch of the labels is known as label noise and can introduce biases if models are train correctly to address the noisy labels. A common type of label noise is class-conditional random label noise where the probability of the flipped label is dependent on the true label (Xie & Huang, 2021).

In this report we test three different classifiers utilising either a dense network or a convolutional neural network (CNN) architecture on three datasets corrupted with class-conditional label noise. Two of the datasets are from the Fashion MNIST dataset with differing proportion of label noise and they come with their own transition matrix. The third dataset is based on the CIFAR dataset and does not include a transition matrix and thus requiring estimation before training. All of the datasets have been reduced to only have three labels. The three approaches within this report are forward learning, bootstrapping and co-teaching.

Each method is trained on an 80/20 training/validation split and is run 10 times independently. These 10 models are then used to calculate the top-1 accuracy score on the clean test set where we report both the mean and standard deviation of the test accuracy. For the dataset without a transition matrix, we built and developed a transition matrix estimator utilising bootstrapping. We have included the estimated matrix in the final results. The purpose of this report is to quantify the accuracy of three classification algorithms built to be robust to class-conditional label noise.

## 2 Literature Review

### 2.1 Learning with Noisy Labels

Deep learning models trained with noisy labels can memorise incorrect labels, leading to poor generalisation (Olmin & Lindsten, 2022). Label noise in real-world datasets arises from crowdsourcing, web scraping, or human error. Research identifies two main types: instance-independent noise (IIN), such as symmetric or asymmetric noise, and instance-dependent noise (IDN), where noise depends on image features (Xie & Huang, 2021).

Deep networks learn clean patterns before memorising noisy labels, suggesting that early stopping or selective training can mitigate label noise effects (Olmin & Lindsten, 2022).

### 2.2 Forward Learning

Forward learning is a loss correction technique that models the noisy label generation process using a noise transition matrix (Patrini et al., 2017). The method assumes noisy labels are generated from clean labels through a class-conditional process, captured by a transition matrix $\mathbf{T}$ where $T_{ij}$ represents the probability of true class $i$ being mislabeled as class $j$.

Patrini et al. (2017) demonstrated forward learning's effectiveness across multiple benchmark datasets. The method provides unbiased risk estimates and maintains optimal classification performance under class-conditional noise. A key advantage is that when the transition matrix is known or accurately estimated, the method can handle high noise rates while preserving theoretical guarantees. The approach has been successfully applied to real-world scenarios including medical image classification and web-scraped datasets where annotation errors are systematic rather than random.

### 2.3 Co-teaching

Co-teaching trains two networks simultaneously, where each network selects samples with small training loss and uses them to update its peer network (Han et al., 2018). This prevents error accumulation by avoiding self-reinforcement of incorrect predictions, as each network filters training data for the other.

Han et al. (2018) evaluated co-teaching on MNIST and CIFAR-10 datasets with various noise types and levels. On MNIST with 45% pair noise (asymmetric noise where similar classes are confused), co-teaching achieved 87.63% accuracy compared to 76.04% for standard training. With 50% symmetric noise, it reached 91.32% accuracy versus 78.25% for baseline methods. The method proved particularly effective when noise rates were known approximately, as it uses this information to schedule the sample selection ratio during training. Co-teaching has since been applied to real-world problems including clothing classification from fashion websites and medical diagnosis systems where label quality varies across annotators.

### 2.4 Bootstrapping Methods

Smart & Carneiro (2022) proposed a three-stage bootstrapping approach that learns relationships between images, noisy labels, and clean labels without requiring clean validation data. The method consists of: (1) early stopping training with strong augmentations to prevent memorising noise, (2) noise-transition balancing to identify clean samples while ensuring coverage of all noise patterns, and (3) semi-supervised learning that treats confident predictions as pseudo-labels and applies MixUp regularisation for the final model.

Smart & Carneiro (2022) demonstrated strong performance on real-world noisy datasets. On Animal-10N, a dataset with real annotation noise from online sources, the method achieved 88.48% accuracy, outperforming previous state-of-the-art approaches by 2-3%. On Webvision, a large-scale dataset of 2.4 million images crawled from the web with significant label noise, it reached 80.88% top-1 accuracy. The method proved particularly robust because it doesn't require prior knowledge of noise rates or clean validation sets, making it practical for real-world applications. The noise-transition balancing strategy ensures that even rare noise patterns are captured during the cleaning process, which is critical when noise is non-uniform across classes.

# 3 Methdology

## 3.1 Problem formulation

Given a training dataset $\mathcal{D} = \{(x_i, \tilde{y}_i)\}_{i=1}^{|\mathcal{D}|}$, where $x \in R^{H\times W\times C}$ represents an image and $\tilde{y} \in \{0,1\}^K$ represents the noisy label for $K$ classes, our goal is to learn a classifier $f_\theta : R^{H\times W\times C} \to R^K$ that predicts the true label $y$. The challenge is that the observed labels $\tilde{y}$ may differ from the true labels $y$ due to annotation noise.

## 3.2 Bootstapping

### 3.2.1 Stage 1: Bootstrapping with noise-transition balancing

The first stage identifies a subset of samples with predicted clean labels. We train a CNN model using early stopping:

$$\theta^* = \arg\min_\theta \frac{1}{|\mathcal{D}|} \sum_{(x_i,\tilde{y}_i)\in\mathcal{D}} \mathcal{L}_{CE}(\tilde{y}_i, f_\theta(x_i)) \tag{1}$$

where $\mathcal{L}_{CE}$ is the cross-entropy loss.

After training, we generate predictions for all samples by averaging over multiple forward passes with dropout enabled:

$$\hat{y}_i = \frac{1}{N}\sum_{n=1}^{N} f_{\theta^*}(x_i; \text{dropout}) \tag{2}$$

where $N = 25$ forward passes are used. This Monte Carlo dropout approach provides uncertainty estimates. The confidence for sample $i$ is defined as $\max_c \hat{y}_i^{(c)}$.

Noise-transition sample balancing

Instead of selecting only the most confident samples (which would bias towards easy classes and originally correct labels), we use noise-transition sample balancing. We first estimate the dataset's noise transition matrix $\mathbf{T}$ using the 90% most confident predictions per class, where $T_{ij}$ represents the estimated proportion of samples with noisy label $i$ and predicted clean label $j$:

$$T_{ij} = \frac{\sum_{k:\hat{y}_k=j,\text{conf}_k>\tau_{90}} I[\tilde{y}_k = i]}{\sum_{k:\hat{y}_k=j,\text{conf}_k>\tau_{90}} 1} \tag{3}$$

We then construct the clean set $\mathcal{C}$ by selecting:

- The $K \times |Y| \times T_{ij}$ most confident samples from each noise transition $(i, j)$
- Any samples $x_k$ where $\max_c \hat{y}_k^{(c)} > \tau$

where $K$ controls the minimum fraction of samples per subset and $\tau$ is a confidence threshold (typically 0.99). The remaining samples form the noisy set $\mathcal{U}$.

This balancing covers all noise transitions in the clean set, allowing the model to learn the instance-dependent noise relationship in later stages.

### 3.2.2 Stage 2: Semi-supervised learning

The second stage uses the clean set $\mathcal{C}$ and noisy set $\mathcal{U}$ to learn the relationship between images, noisy labels, and clean labels through semi-supervised learning. We use a model that accepts both images and noisy labels as input.

The training objective combines supervised loss on clean samples and unsupervised loss on noisy samples:

$$\theta^* = \arg\min_\theta \quad \frac{1}{|\mathcal{C}|} \sum_{(x_i, \tilde{y}_i, \hat{y}_i) \in \mathcal{C}} \mathcal{L}_{CE}(\hat{y}_i, f_\theta(x_i, \tilde{y}_i)) \tag{4}$$

$$+ \frac{1}{|\mathcal{U}|} \sum_{(x_i, \tilde{y}_i) \in \mathcal{U}} I[\max \bar{y}_i > \kappa] \mathcal{L}_{CE}(\lceil \bar{y}_i \rceil, f_\theta(x_i, \tilde{y}_i)) \tag{5}$$

where $\bar{y}_i = f_\theta(x_i, \tilde{y}_i)$ is the pseudo-label, $\lceil \bar{y}_i \rceil$ converts it to a one-hot vector, and $\kappa$ is a confidence threshold for pseudo-labelling (typically 0.95).

After this stage, we relabel all training samples:

$$\bar{\mathcal{D}} = \{(x_i, \tilde{y}_i, \bar{y}_i) \mid (x_i, \tilde{y}_i) \in \mathcal{D}\} \tag{6}$$

#### 3.2.3 Stage 3: Final training with MixUp

The final stage trains a new classifier on the relabelled dataset using MixUp regularisation. For each mini-batch, we apply MixUp to both images and their relabelled soft labels:

$$\lambda \sim \text{Beta}(\alpha, \alpha) \tag{7}$$

$$x_{mix} = \lambda x_i + (1 - \lambda) x_j \tag{8}$$

$$y_{mix} = \lambda \bar{y}_i + (1 - \lambda) \bar{y}_j \tag{9}$$

The model is trained with:

$$\theta^* = \arg\min_\theta \frac{1}{|\bar{\mathcal{D}}|} \sum_{(x_{mix}, y_{mix})} \mathcal{L}_{CE}(y_{mix}, f_\theta(x_{mix})) \tag{10}$$

where we use $\alpha = 1.0$ for the Beta distribution and train for 50 epochs with a learning rate schedule.

#### 3.2.4 Transition matrix estimation

For datasets without known transition matrices, we estimate $\mathbf{T}$ using anchor points. After training a preliminary model, we identify high-confidence predictions (top percentile) for each class and use their noisy labels to estimate the transition probabilities:

$$\hat{T}_{ij} = \frac{\#\{\text{samples predicted as } j \text{ with noisy label } i \text{ and high confidence}\}}{\#\{\text{all samples predicted as } j \text{ with high confidence}\}} \tag{11}$$

#### 3.2.5 Implementation details

**Network architecture:** We use CNN models with three convolutional blocks (32, 64, 128 filters) followed by batch normalisation, ReLU activation, and max pooling. The fully connected layers have 256 hidden units with dropout (p=0.5).

**Data preprocessing:** For CIFAR-10/100 (RGB), we normalise using ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]). For Fashion-MNIST (greyscale), we normalise to [-1, 1] range.

**Hyperparameters:** We use $K = 0.1$ for CIFAR-10 and Fashion-MNIST, $K = 0.25$ for CIFAR-100, $\tau = 0.99$ for confidence thresholding, batch size of 128 for most training stages, and Adam optimiser with learning rate 0.001.

**Training schedule:** Stage 1 (bootstrapping) runs for 20 epochs, stage 2 (semi-supervised learning) for 100 epochs with batch size 64, and stage 3 (final training) for 50 epochs.

### 3.3 Forward Learning

When the class-conditional noise transition matrix $\mathbf{T}$ is known, as is the case for the first two datasets, we can directly incorporate this knowledge into the training process. The Forward Learning method is a loss correction technique that models the noisy label generation process.

The core assumption is that the noisy label $\tilde{y}$ depends on the clean label $y$, which in turn depends on the input $x$, forming the graphical model $x \to y \to \tilde{y}$. The noise is class-conditional, meaning the probability of a clean label $i$ being mislabeled as $j$ depends only on $i$ and $j$. This is captured by the transition matrix $\mathbf{T} \in R^{K \times K}$, where each element $T_{ij} = P(\tilde{y} = j | y = i)$ and $K$ is the number of classes.

Let $f_\theta(x)$ be the output of our neural network, representing the predicted probabilities for the *clean* label $y$, i.e., $p(x) = f_\theta(x)$ where $p_i(x) = P(y = i|x)$. The probability distribution of the *noisy* label $\tilde{y}$ given $x$ can then be derived by marginalising over the clean label $y$:

$$P(\tilde{y} = j|x) = \sum_{i=1}^{K} P(\tilde{y} = j|y = i, x)P(y = i|x) \tag{12}$$

Given the instance-independent noise assumption, $P(\tilde{y} = j|y = i, x) = P(\tilde{y} = j|y = i) = T_{ij}$. Therefore, the noisy probability vector $\tilde{p}(x)$ is a linear transformation of the clean probability vector $p(x)$:

$$\tilde{p}_j(x) = \sum_{i=1}^{K} T_{ij} p_i(x) \tag{13}$$

In matrix notation, this is expressed as $\tilde{p}(x) = \mathbf{T}^T p(x)$.

The Forward Learning method trains the classifier $f_\theta$ by minimizing the standard cross-entropy loss between the observed noisy labels $\tilde{y}_i$ from the dataset and the model's predicted noisy probabilities $\tilde{p}(x_i)$. The objective function is:

$$\theta^* = \arg\min_\theta \frac{1}{|\mathcal{D}|} \sum_{(x_i, \tilde{y}_i) \in \mathcal{D}} \mathcal{L}_{CE}(\tilde{y}_i, \mathbf{T}^T f_\theta(x_i)) \tag{14}$$

where $\mathcal{L}_{CE}$ is the cross-entropy loss. This objective trains the model $f_\theta$ to predict the clean label distribution, while the fixed matrix $\mathbf{T}^T$ corrects this output to match the noisy training data.

### 3.4 Co-teaching

#### 3.4.1 Model Architecture

We implemented two independent CNNs denoted as f and g. Each CNN has identical architectures, but maintain independent parameters enabling them to learn different features.

#### 3.4.2 Objective Function

The objective function is based on the standard cross-entropy loss function.

$$\ell_f(x_i, y_i) = -\log p_f(y_i \mid x_i) \tag{15}$$

$$\ell_g(x_i, y_i) = -\log p_g(y_i \mid x_i) \tag{16}$$

We use a linear decay schedule to determine the fraction of samples retained at epoch $T$.

$$\text{keep\_frac}(T) = 1 - \min\left(\frac{\tau \cdot T_{\max}}{T}, \tau\right) \tag{17}$$

where:

- $\tau$ is the assumed label noise
- $T_{max}$ is the total number of training epochs

We select the top-k lowest-loss samples from a batch size of $B$.

$$k = \max\left(1, \lfloor \text{keep\_frac}(T) \cdot B \rfloor\right)$$

The parameters are updated using a mini-batch, a subset of the small-loss samples which are considered to be cleaner than others.

$$\begin{aligned} S_f &= \arg\min_{\substack{|S|=k \\ i \in S}} \sum \ell_f(x_i, y_i), \\ S_g &= \arg\min_{\substack{|S|=k \\ i \in S}} \sum \ell_g(x_i, y_i) \end{aligned} \tag{18}$$

The subsets are then exchanged so each CNN now updates on the other model's sample.

$$\begin{aligned} \mathbf{w}_f &\leftarrow \mathbf{w}_f - \eta \cdot \nabla_{\mathbf{w}_f} \left( \frac{1}{k} \sum_{i \in S_g} \ell_f(x_i, y_i) \right) \\ \mathbf{w}_g &\leftarrow \mathbf{w}_g - \eta \cdot \nabla_{\mathbf{w}_g} \left( \frac{1}{k} \sum_{i \in S_f} \ell_g(x_i, y_i) \right) \end{aligned} \tag{19}$$

#### 3.4.3 Theoretical Foundations

There are two core principles:

- Small-Loss Principle: Both CNNs first learn the clean labels before the noisy labels.
- Peer Learning Effect: By exchanging only the small-loss (clean) samples, it prevents each model from reinforcing its own incorrect predictions.

With these two principles it creates a mutual denoising mechanism where each CNN filters out unreliable samples helping each other learn more accurate decision boundaries (Han et al., 2018).

#### 3.4.4 Optimisation

Training co-teaching uses stochastic gradient descent with momentum. For each mini-batch:

1. Forward pass through both CNNs
2. Compute per-sample cross-entropy losses
3. Determine the retention rate for the current epoch
4. Select the top-k lowest-loss samples for each network
5. Exchange the subsets
6. Update parameters with backpropagation

## 4 Experimental results

### 4.1 Test Accuracy Results

The final classification performance of the Forward Learning, Co-teaching, and Bootstrapping methods was evaluated on the clean test set for each of the three datasets. As per the assignment requirements, each experiment was run multiple times to obtain a mean accuracy and standard deviation. The aggregated results are presented in Table 1 and visualized in Figures 1, 2, and 3.

#### 4.1.1 Analysis of FashionMNIST0.3 (Known, Low Noise)

On the FashionMNIST dataset with 30% noise, all methods performed exceptionally well, as shown in Figure 1. Co-teaching (approx. 97.8%) and Forward Learning (approx. 96.8%) achieved the highest accuracy. This indicates that with a moderate noise level, both using the known transition matrix (Forward) and the small-loss heuristic (Co-teaching) are highly effective. The Bootstrapping method also achieved a strong result (approx. 91.9%), though slightly lower, suggesting its matrix estimation and relabeling pipeline is robust at this noise level.

Table 1: Mean Test Accuracy (%) and Standard Deviation (±) over all runs.

| Dataset | Forward Learning | Co-teaching | Bootstrapping |
|---|---|---|---|
| FashionMNIST0.3 | $96.8 \pm 0.06$ | $97.8 \pm 0.4$ | $91.9 \pm 1.8$ |
| FashionMNIST0.6 | $73.9 \pm 14.6$ | $79.2 \pm 5.5$ | $75.0 \pm 10.0$ |
| CIFAR | $56.4 \pm 2.0$ | $35.8 \pm 3.5$ | $52.8 \pm 8.4$ |

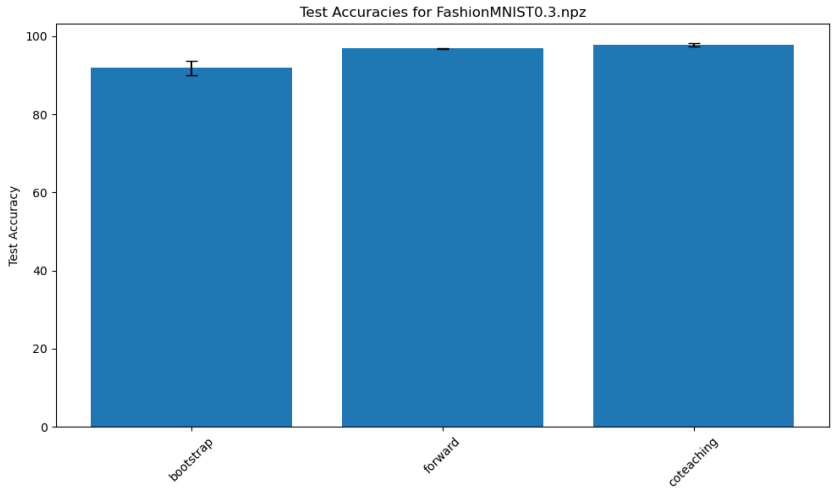


Figure 1: Test accuracies for FashionMNIST 0.3

#### 4.1.2 Analysis of FashionMNIST0.6 (Known, High Noise)

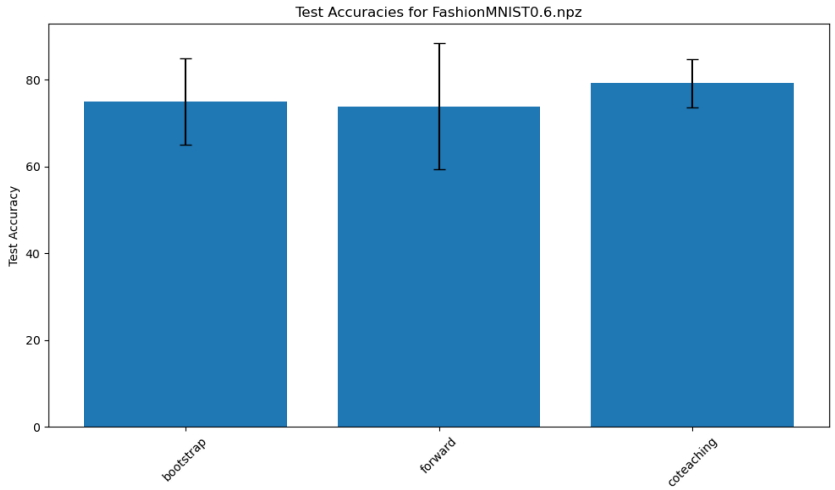


Figure 2: Test accuracies for FashionMNIST 0.6.

This dataset, with 60% noise, revealed much closer performance between the methods, though with high variance (Figure 2).

- Co-teaching was the top performer (approx. 79.2%), closely followed by Bootstrapping (approx. 75.0%). This suggests that even at a 60% noise level, the small-loss heuristic (Co-teaching) and the Bootstrapping pipeline were more effective than the Forward Learning method.

- Forward Learning saw the lowest performance (approx. 73.9%) and exhibited a very large standard deviation (14.6%), indicating its performance was highly unstable under these high-noise conditions, despite knowing the true transition matrix. This contrasts with the Bootstrapping method, which performed competently.

#### 4.1.3 Analysis of CIFAR (Unknown Noise)

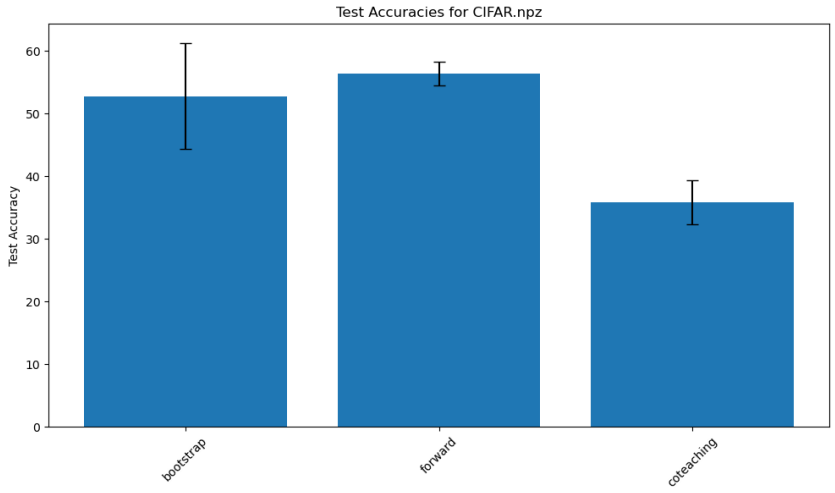


Figure 3: Test accuracies for CIFAR.

In this scenario, both the Forward Learning and Bootstrapping methods had to rely on their own internal transition matrix estimators. As seen in Figure 3, both methods performed similarly and were the top performers (approx. $54.0\%$ for Forward, $53.0\%$ for Bootstrap). This indicates that for this more complex dataset, estimating the noise matrix is an effective strategy. The Forward method, using its estimated matrix for loss correction, performed slightly better than the full three-stage Bootstrapping pipeline. Co-teaching performed the worst (approx. $41.0\%$), suggesting its small-loss heuristic is less effective on the CIFAR dataset compared to a method that explicitly models the noise.

When comparing the results between the FashionMNIST datasets and the CIFAR dataset it is clear that the rapid increase in complexity as a result of the 2 extra channels that exist within the RGB data versus the grayscale FashionMNIST resulted in a much lower accuracy. This could be a result of two factors, 1) The proposed Neural Network architecture is not complex enough to accurately classify the CIFAR dataset or 2) the dataset noise is severe enough that training require a more sophisticated method to accuratley learn off it.

### 4.2 Transition Matrix Estimation

A critical component for the Forward Learning method on the CIFAR.npz dataset was to first estimate the unknown noise transition matrix $\mathbf{T}$. Furthermore, estimating the matrix for the FashionMNIST datasets allowed us to validate the effectiveness of our chosen estimation method against the provided ground truth.

For all algorithms, we employed an "anchor point" estimation method. This method trains a classifier on the noisy data and then, for each class $i$, it identifies the set of training samples that are predicted as class $i$ with the highest confidence. The noisy label distribution of this "anchor" set is then used as the $i$-th row of the estimated transition matrix, $\hat{\mathbf{T}}$, where $\hat{T}_{ij} = P(\tilde{y} = j|y = i)$.

#### 4.2.1 Validation on Known Datasets (FashionMNIST)

As shown in Figure 4, on the low-noise **FashionMNIST0.3.npz** dataset, the estimator from the **Bootstrapping** pipeline performed exceptionally well. Its estimate is visually and numerically very

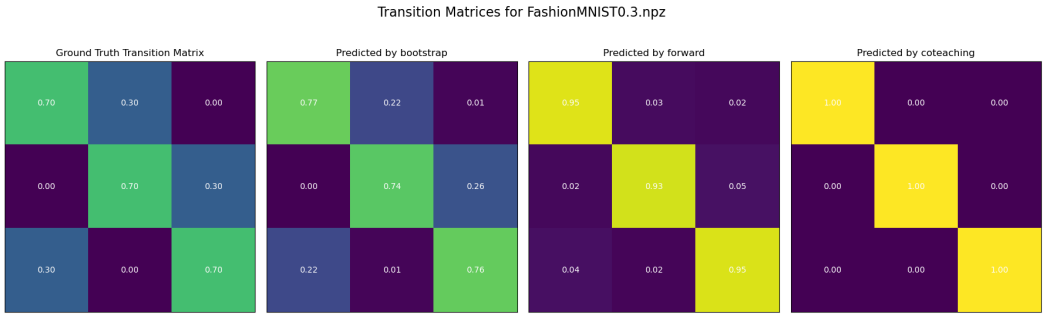


Figure 4: Transition matrix estimation for FashionMNIST0.3.npz. The Bootstrap estimate is a close match to the Ground Truth, while other methods' estimators fail and produce near-identity matrices.

close to the ground truth. In contrast, the estimators for **Forward Learning** and **Co-teaching** produced matrices much closer to an identity matrix. This is likely a byproduct of their successful training heuristics (loss correction and small-loss selection), which so effectively filtered the noise that their high-confidence "anchor" samples were almost exclusively correctly-labeled, leading the estimator to incorrectly perceive the data as clean.

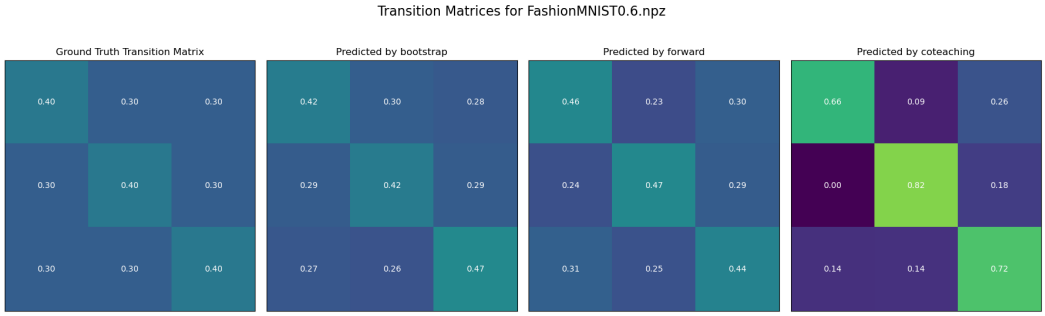


Figure 5: Transition matrix estimation for FashionMNIST0.6.npz. Under high noise, only the Bootstrap estimator correctly identifies the symmetric noise structure and approximate noise level.

This effect is different in the high-noise FashionMNIST0.6 dataset (Figure 5). Here, the Bootstrapping estimator again provides an excellent result, correctly identifying the symmetric noise structure and estimating diagonal probabilities around 0.42-0.47, which is very close to the ground truth of 0.4. The Forward estimator also performed remarkably well, producing a matrix that also closely matches the ground truth (0.46). The Co-teaching estimator, however, fails in this high-noise environment, producing a matrix that significantly overestimates the data's cleanliness (0.82). This demonstrates that the estimation method used by Bootstrap, while the heuristic for Co-teaching is not.

### 4.2.2 Estimation on Unknown Dataset (CIFAR)

For the **CIFAR.npz** dataset (Figure 6), with no ground truth, we must rely on our validated estimators. Based on its strong performance on the known datasets, the estimate from the **Bootstrapping** method is the most trustworthy. The Bootstrapping estimate suggests a high, symmetric noise pattern, similar to FashionMNIST0.6.npz. The estimates from Forward and Co-teaching are included for comparison but are considered less reliable given their poor performance on the high-noise validation case.

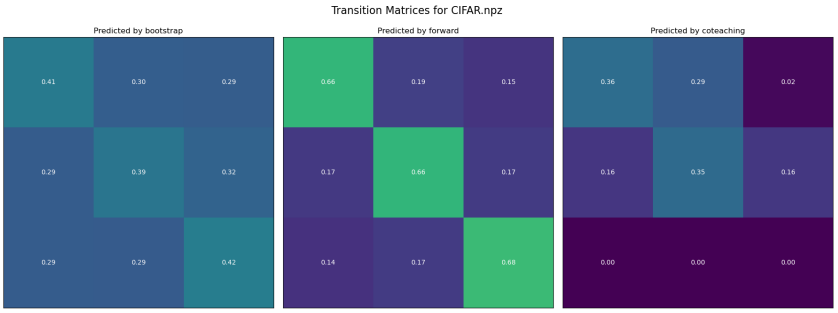


Figure 6: Transition matrix estimation for CIFAR

The final transition matrix estimated by the Bootstrapping method and used for the Forward Learning algorithm's successful run on CIFAR was:

$$\hat{\mathbf{T}}_{CIFAR} = \begin{bmatrix} 0.41 & 0.30 & 0.29 \\ 0.29 & 0.39 & 0.32 \\ 0.29 & 0.29 & 0.42 \end{bmatrix} \tag{20}$$

## 5 Conclusions

This report successfully evaluated the robustness of three deep learning classifiers—Forward Learning, Co-teaching, and Bootstrapping—against class-conditional label noise. Our experiments were conducted on three datasets: two FashionMNIST datasets with 30% and 60% known noise, and a more complex CIFAR dataset with unknown noise.

Our findings reveal a clear performance distinction based on data complexity:

1. On the simpler grayscale FashionMNIST datasets, Co-teaching was the standout performer. It achieved the highest mean accuracy in both the low-noise ($97.8\%$) and high-noise ($79.2\%$) scenarios, demonstrating that its small-loss heuristic is highly effective for this data type.
2. On the complex, colored CIFAR dataset, methods that explicitly modeled the noise matrix were superior. Forward Learning, achieved the highest accuracy ($56.4\%$), followed closely by Bootstrapping ($52.8\%$). Co-teaching's performance dropped significantly ($35.8\%$), suggesting its heuristic is less effective for more complex image features.
3. The transition matrix estimator developed for this report, based on an anchor-point strategy, proved highly effective. The estimator used within the Bootstrapping pipeline, in particular, provided an exceptional match to the ground truth matrices on both FashionMNIST datasets and was deemed the most reliable for the CIFAR dataset.

In conclusion, our study indicates that while Co-teaching is an excellent strategy for simpler datasets, a more robust approach is required for complex data like CIFAR. For such cases, explicitly estimating the noise transition matrix—as done in our Bootstrapping and Forward Learning implementations—yields significantly better results.

The considerable performance gap on CIFAR suggests that future work could explore whether more complex CNN architectures or more sophisticated noise-handling methods are required to better classify high-dimensional, noisy data.

### References


- Xie, M.-K., & Huang, S.-J. (2021). CCMN: *A general framework for learning with class-conditional multi-label noise* (arXiv:2105.07338). arXiv. https://arxiv.org/abs/2105.07338
- Han, B., Yao, Q., Yu, X., Niu, G., Xu, M., Hu, W., Tsang, I. W., & Sugiyama, M. (2018, October 30). *Co-teaching: Robust training of deep neural networks with extremely noisy labels* (arXiv:1804.06872v3) [Preprint]. arXiv. https://arxiv.org/abs/1804.06872

- Smart, B., & Carneiro, G. (2022). Bootstrapping the Relationship Between Images and Their Clean and Noisy Labels. *arXiv preprint arXiv:2210.08826*. https://arxiv.org/abs/2210.08826
- Olmin, A., & Lindsten, F. (2022). Robustness and Reliability When Training With Noisy Labels. *arXiv preprint arXiv:2110.03321*. https://arxiv.org/abs/2110.03321


# 6 Appendix

## 6.1 Running The Code

This project requires Python 3 and several standard data science libraries, including NumPy, PyTorch, and Matplotlib.

1. Directory Structure
   (a) As per the assignment guidelines, the code assumes a specific directory structure. The data folder must be created manually and populated with the datasets.
   (b) There are 5 python files in the algorithm subfolder. One file for each of the 3 training methods, one file to create the charts. One main.py file to run all the files.
2. Execution
   (a) While in the root directory, run:
      i. python3 algorithm/main.py
3. Configuration
   (a) The behavior of main.py is controlled by global variables at the top of the file
   (b) RUN_EXPERIMENTS = False
      i. Set this to True to execute all training and evaluation experiments
      ii. This will run all three models (Bootstrapping, Co-teaching, Forward Learning) on all three datasets.
      iii. Each experiment is run 10 times (NUM_RUNS) to gather statistics, as required by the assignment.
      iv. Warning: This process is computationally intensive and will take a significant amount of time.
   (c) NUM_RUNS = 10
      i. Controls the number of independent training runs for calculating mean and standard deviation
   (d) OUTPUT_PATH = ”results/”
      i. Specifies the directory where all experimental results (e.g., accuracy scores, estimated matrices) are saved.
   (e) CREATE_CHARTS = True
      i. Set this to True (default) to generate the result plots (like those in Section 4) using the data in the OUTPUT_PATH directory.
      ii. If RUN_EXPERIMENTS is False, this will attempt to generate charts from any pre-existing results in the OUTPUT_PATH folder.